



為實現遊戲系統所需之運算與儲存功能，本架構進行了三項主要硬體設計優化。首先，將資料記憶體（Data Memory）容量擴充至 4 KB，並自 CPU 核心內部獨立配置於頂層模組，以利系統進行全域記憶體映射輸入輸出（Memory-Mapped I/O, MMIO）之位址解碼與周邊資源管理。其次，為降低資料通路之硬體複雜度並縮短關鍵路徑，將原微架構中獨立的 Shift Left One 單元整合至立即數產生器（Immediate Generator, ImmGen）模組內部，以簡化訊號傳遞流程並提升時序效能。

此外，系統透過 MMIO 機制整合七段顯示器控制器（Seven Seg Controller）與按鈕防彈跳（Debounce）模組。其中，七段顯示器負責遊戲資訊與系統狀態之即時顯示，而防彈跳模組則用於消除機械按鈕切換時所產生之訊號抖動，確保輸入訊號的穩定性與可靠性。藉由上述軟硬體整合設計，本系統成功建立一個具備處理器運算能力、周邊控制功能以及互動式遊戲應用之 FPGA SoC 平台

### III. RISC-V SOFTWARE IMPLEMENTATION

本專題之核心遊戲邏輯完全由 RISC-V 組合語言編譯之機器語言驅動。為了提升系統的可讀性與模組化程度，軟體架構依據實體開關所切換的四大模式（Mode 00/01/10/11），將程式流程切分為四大核心區塊。以下針對各區塊之軟體演算法、暫存器配置與實體按鈕互動進行詳細說明：

#### A. 遊戲初始化與開局謎底生成

當系統啟動時，玩家需先將最左側控制開關切換至啟用狀態，使系統進入待機模式。此時七段顯示器將閃爍顯示訊息 (1A2b)，引導玩家按下【中鍵】開始遊戲。

在等待玩家按下【中鍵】的期間，處理器持續執行偽隨機數產生器（Pseudo-Random Number Generator, PRNG）。由於使用者按下按鈕的時刻具有不可預測性，系統於主輪詢迴圈（Polling Loop）中以 FPGA 100 MHz 系統時脈持續更新線性回饋移位暫存器（Linear Feedback Shift Register, LFSR）之狀態值。當偵測到【中鍵】被按下時，處理器立即擷取當前 LFSR 內容作為亂數種子，並透過除法、取餘數及重複性檢查等運算流程，篩選出四個互不重複的十進位數字，依序儲存於答案暫存器（s4 至 s7）中，完成遊戲謎底之動態生成。

謎底產生完成後，系統進一步執行遊戲環境初始化程序，包括堆疊指標、猜測次數計數器以及相關全域暫存器之設定，並建立歷史紀錄儲存區的初始狀態。完成初始化後，系統即進入正式遊戲階段，等待玩家輸入第一組猜測數字。

#### B. 一般猜測模式與 A/B 判定邏輯（Mode 00）

謎底生成後，系統進入 Mode 00 的主要遊戲循環。玩家可以透過實體按鈕即時編輯欲猜測的數字，並由 CPU 進行合法性檢查與結果判定：

1. **數值編輯：**游標預設於最高位。玩家按下【左鍵】或【右鍵】可切換當前欲修改的七段顯示器欄位；按下【上鍵】或【下鍵】則會對該位數暫存器進行 +1 或 -1 的 0~9 循環修改，並透過 MMIO 即時更新七段顯示器畫面。
2. **合法性檢查：**當玩家調整完四位數並按下【中鍵】確認送出時，CPU 首先會執行輸入判定。若偵測到玩

家輸入了重複的數字或開頭為 0，畫面將強制鎖定並閃爍顯示錯誤代碼 CEDD (Err)，拒絕本次猜測，直到玩家修正。

3. **A/B 數量統計與堆疊：**若輸入合法，CPU 將啟動全域比對電路，將輸入暫存器與答案暫存器進行交叉比對。位置與數字皆正確者增加 A 的計數；數字正確但位置錯誤者增加 B 的計數。判定完成後，總猜測次數自增一次。隨後，CPU 執行暫存器堆疊寫入指令，並顯示結果在畫面中。

#### C. 猜測次數統計與作弊除錯模式（Mode 01 & Mode 11）

為了提供多維度的遊戲回饋與開發除錯便利性，系統利用開關控制切換至次要輔助模式：

1. **次數顯示（Mode 01）：**當玩家將實體指撥開關撥至 01 時，CPU 將暫停 Mode 00 的編輯畫面，改為讀取內部計數暫存器，並將目前為止累積的「成功猜測總次數」轉為 BCD 碼映射至七段顯示器上，方便玩家評估目前的遊戲進度。

2. **除錯模式（Mode 11）：**為避免除錯功能於一般操作過程中被誤觸發，本系統對原始設計進行安全性優化。原規劃僅以最後兩個開關組合（2'b11）作為除錯模式的啟動條件，使用者在切換歷史紀錄模式（2'b10）時，可能因操作失誤而意外進入除錯模式。為提升系統可靠性，本設計改採四位元全開（4'b1111）安全互鎖機制作為除錯模式的唯一啟動條件。

當 CPU 讀取開關控制暫存器（0x4000000C）時，會同步檢查四個指撥開關的狀態。僅當所有開關皆處於啟用狀態（SW[3:0] = 4'b1111）時，系統才會進入除錯模式，並將儲存在答案暫存器中的四位謎底數字輸出至七段顯示器，以供開發與驗證使用。

若開關組合為其他狀態，例如 4'b0011、4'b0111 或其他非 4'b1111 的組合，系統將視為無效模式（Invalid Mode），維持原有顯示內容或執行保護性隱藏機制，不會揭露遊戲答案。透過此設計，系統能有效避免因操作失誤而導致謎底曝光，同時提供開發過程與驗證階段快速檢查偽隨機數產生器與遊戲邏輯正確性的途徑。

#### D. 歷史紀錄查詢功能（Mode 10）

當切換至 Mode 10 時，程式將啟動核心的暫存器堆疊動態瀏覽機制。系統將暫存器 a7 鎖定為最新的堆疊指標位置，並允許玩家使用按鈕自由翻閱 Data Memory 中的所有歷史猜測：

1. **切換：**在瀏覽歷史紀錄時，玩家按下【中鍵】可對標記 t3 進行 0 與 1 的狀態翻轉，實現七段顯示器在「猜測數字」與「猜測結果」之間的即時畫面切換
2. **翻頁：**玩家按下【上鍵】時，程式在進行邊界安全檢查（未超越初始堆疊邊界 0xFF8）後，驅動 sp 指標向舊資料方向移動；按下【下鍵】時，在確保未超越最新資料邊界（a7）的前提下，驅動 sp 指標向新資料移動。且每次換頁成功時，狀態標記 t3 強制歸零，確保優先呈現該筆紀錄的「猜測數字」。

#### IV. VERIFICATION & TESTING RESULTS

將最終產生的 Bitstream 導入 FPGA 中，並進行功能測試：

##### A. 實作與測試結果

TABLE I. TEST

| Case | 測試情境         | 預期輸出結果                                     | 實際測試結果                                    | 判定   |
|------|--------------|--------------------------------------------|-------------------------------------------|------|
| 1    | 謎底初始化與除錯功能測試 | 除錯功能正常運作，並成功產生符合要求之謎底並且未重複。                | 重複打開開關5次並進入除錯模式查看謎底，皆為符合要求之謎底並且不重複。       | PASS |
| 2    | 左右切換功能測試     | 會根據左右鍵切換編輯目標，並且最左邊往左會變做右邊，最右邊往右會變最左邊。      | 無論是在邊界還是在中間，都能依照預期做切換。                    | PASS |
| 3    | 數字編輯測試       | 使用上下鍵修改數值，預期按上數值會增加，按下則數值減少，並且9+1=0，0-1=9。 | 無論是在邊界還是在中間，都能依照預期編輯數字。                   | PASS |
| 4    | 猜測結果測試       | 根據除錯模式得到的謎底，給出正確的結果。                       | 按照謎底4801輸入猜測1237得到0A1b正確判斷一個數字正確但位置不正確。   | PASS |
| 5    | 次數顯示測試       | 切換至次數顯示模式，顯示猜測次數。                          | 正確顯示猜測次數(2)。                              | PASS |
| 6    | 歷史查詢測試       | 切換至歷史查詢模式，會先顯示上次猜測，按中鍵時會顯示猜測結果，按上下鍵切換查詢目標。 | 正常顯示歷史紀錄。                                 | PASS |
| 7    | 正確答案測試       | 送出正確答案，交替顯示4A與猜測次數。                        | 送出正解4801後，顯示器交替顯示4A與猜測次數(3)。              | PASS |
| 8    | 違規猜測         | 如果送出的猜測包含重複數字或者為0開頭，顯示Err。                 | 送出重複數字(2234)，顯示Err，送出0開頭猜測(0234)，一樣顯示Err。 | PASS |
| 9    | 重製測試         | 在正確之後按中鍵重新開始，歷史紀錄與猜測次數被正確清空。               | 歷史紀錄與猜測次數被清空。                             | PASS |

成果展示影片連結：

[https://youtu.be/x1N\\_rKJL9kM?si=fvMqYfRA33y\\_zNe1](https://youtu.be/x1N_rKJL9kM?si=fvMqYfRA33y_zNe1)

##### B. 問題與除錯

在實作過程中所遭遇的最大硬體挑戰在於 Instruction Memory 與處理器程式執行之間的時序耦合問題。由於系統啟動或重設釋放的瞬間，CPU 的取指執行與 Instruction Memory 的加載同步開始，導致第一行指令在尚未完全加載完成前處理器便已開始解碼執行，造成該條指令形同被系統「吞噬」。由於本程式的第一行指令恰好是用於初始化與存放七段顯示器 MMIO 映射位址的關鍵核心指令，這使得系統在開局時便陷入位址指標污染的狀態，在缺失螢幕顯示的情況下，完全無法找出問題所在。然而，在先前的獨立周邊測試中，不論是按鈕、開關或七段顯示器的 I/O 驗證皆能完全正常運作，經深入比對程式碼後發現，該 I/O 測試程式內部包含一個完整的無限迴圈，使得初始化與映射指令有機會被重複循環執行；以此為關鍵線索，懷疑是常規遊戲程式在實機部署時發生指令遺失所致。最終，透過在軟體程式開頭主動加入一行無實質影響的「多餘緩衝指令（NOP）」，成功確保了後續初始化邏輯的加載與執行，順利攻克此項實機部署的時序瓶頸。

#### V. CONCLUSION & APPENDIX

##### A. 專題結論與成果總結

本期末專題成功於 Basys 3 FPGA 開發板上建構了一台「基於 RISC-V 核心之 1A2B 互動式猜數字遊戲機」。藉由軟硬體協同設計（SoC）的架構思維，將標準的單

週期 RISC-V 處理器核心與外部擴充的 4KB 資料記憶體、七段顯示器控制器及硬體按鈕防彈跳模組進行全域的記憶體映射 I/O（MMIO）縫合。在軟體層面，透過編譯之組合語言程式，不僅實現了高隨機性的 PRNG 謎底生成、單週期全域 A/B 判定演算法，更利用 Data Memory 的有限空間開發出高安全性的動態暫存器堆疊機制，賦予系統流暢的歷史紀錄翻閱功能。本系統之實機測試結果展現了極高的反應靈敏度與運算正確性，完美印證了軟體控制靈活性與硬體時脈驅動高度分工之軟硬體整合設計精髓。

##### B. 外部來源與開源專案使用說明

本專題之單週期微處理器核心係繼承並參考自 NYCU CO2026 Lab3 之單週期處理器設計藍本，並在該基礎架構上進行了自主優化與核心修改，包含：

1. 將原微架構中獨立的左移單元移出並直接整合至立即數產生器（ImmGen）內部，精簡了關鍵路徑之走線延遲。
2. 建立頂層模組（top\_soc.v），自主設計並實作了基於地址解碼器（Address Decoder）與多工器（Mux）之全域 MMIO 總線結構。
3. 完全獨立編寫 1A2B 遊戲之 RISC-V 組合語言程式，並導入了利用 sp 指標控制的動態堆疊歷史資料翻閱演算法。

##### C. AI 協作紀錄摘要

| 使用AI的地方    | 我問了什麼                  | AI提供什麼協助                                                          | 我如何檢查與修改                  |
|------------|------------------------|-------------------------------------------------------------------|---------------------------|
| Verilog輔助  | 根據我的架構給出修正意見。          | 給出修正意見。                                                           | 參考他的意見進行修改，並反覆嘗試直到如預期運作。  |
| Vivado如何使用 | Vivado如何使用?            | 給出操作提示。                                                           | 根據提示操作，如產生最終Bitstream的流程。 |
| 程式輔助       | 為何在歷史查詢模式，按鍵操作會比正常模式卡? | 指出我在歷史查詢模式中，判斷按鍵鬆開的指令出現在錯誤位置，導致在我未按按鍵的情況下要求我放開按鍵，導致無法進入檢查有按按鍵的指令。 | 調整指令，必須在有按按鍵的情況下才檢查是否鬆開。  |
| 報告潤色       | 以上是我的內容，幫我潤色一下。        | 潤色我給他的文字。                                                         | 篩選出我需要的內容，複製到報告中。         |

##### D. 個人貢獻說明

本專題由本人獨立完成所有設計與整合工作。本人的實際核心貢獻包括：獨自構思並規劃 1A2B 遊戲機的互動機制；自主學習 Vivado 開發工具的操作、XDC 引腳與 MMIO 位址的對應綁定；獨立測試並修改 Verilog 初始代碼以使其適應 Basys 3 開發板之硬體規範；完全獨立編寫整套 1A2B 遊戲之 RISC-V 組合語言程式（包含動態 LFSR 亂數、A/B 比對統計、Mode 11 安全遮罩及 Mode 10 堆疊翻頁邏輯）；以及獨立主導實機除錯、排除首條指令遺失之時序瓶頸，並撰寫本篇成果結案報告。

#### E. 未完成項目與未來改善

雖然本專題已在有限的時間與技術背景下，成功透過軟硬體協同設計實現了完整且具備安全互鎖機制的 1A2B 猜數字遊戲，但未來仍有以下工程優化空間：

1. 硬體描述語言（Verilog）之底層重構：受限於對 HDL 語言的不熟悉，頂層周邊控制電路多仰賴 AI 優化與密集實機測試。未來可規劃深入修讀系統晶片或數位電路相關課程，將現有之 MMIO 總線與解碼器重構為更符合業界標準之 AMBA APB 或 AXI-Lite 總線協議，以提升核心之擴充性。
2. 中斷驅動架構：目前按鈕與模式切換皆採用 CPU 軟體輪詢版本，若未來能將實體按鈕觸發整合至 RISC-V 的非同步中斷控制，將能徹底釋放 CPU 的運算資源。

#### F. GitHub 專案倉庫連結

專案 GitHub 連結：<https://github.com/cCbillCc/1A2B-game-console>

#### REFERENCES

- [1] National Yang Ming Chiao Tung University (NYCU), "Lab 3: Single Cycle CPU," Course Project for CO2026 Computer Organization, [Online].  
Available: <https://github.com/nycu-caslab/CO2026/tree/main/Lab3>